

GIT И GITHUB

ИНТЕРАКТИВНЫЙ ГАЙД.....	2
Модуль 1: Базовые концепции и локальная работа.....	2
1.1 Чек-лист знаний.....	2
1.2 Глоссарий.....	2
1.3 Полезные ссылки.....	2
1.4 Задача на проверку.....	3
Модуль 2: Ветвление и Git Flow.....	4
2.1 Чек-лист знаний.....	4
2.2 Глоссарий.....	4
2.3 Полезные ссылки.....	4
2.4 Задача на проверку.....	4
Модуль 3: Удаленные репозитории и GitHub.....	6
3.1 Чек-лист знаний.....	6
3.2 Справочная таблица основных команд.....	6
3.3 Полезные ссылки.....	7
3.4 Задача на проверку.....	7
СБОРНИК ПРОВЕРОЧНЫХ ТЕСТОВ.....	8
Тест 1: Локальный Git.....	8
Часть А: Выберите один правильный ответ.....	8
Часть Б: Задания со свободным ответом (команды).....	9
Тест 2: Ветвление и конфликты.....	10
Часть А: Выберите один правильный ответ.....	10
Часть Б: Задания со свободным ответом (команды).....	11
Тест 3: Работа с GitHub и совместная разработка.....	12
Часть А: Выберите один правильный ответ.....	12
Часть Б: Задания со свободным ответом (команды).....	13
ОТВЕТЫ И РАЗБОР ЗАДАНИЙ.....	14

Тест 1: Локальный Git.....	14
Тест 2: Ветвление и конфликты.....	15
Тест 3: Работа с GitHub и совместная разработка.....	16

ИНТЕРАКТИВНЫЙ ГАЙД

Здесь всё, что нужно знать, помнить и уметь применять на автомате. Если вы что-то забыли – это ваш чек-лист для самопроверки и ликвидации пробелов.

Модуль 1: Базовые концепции и локальная работа

Что нужно уметь: Инициализировать репозиторий, управлять тремя зонами Git (рабочая директория, индекс, коммит), просматривать историю изменений и правильно фиксировать версии кода.

1.1 Чек-лист знаний

- [] Понимание разницы между Git (распределенная система контроля версий) и GitHub (хостинг репозитория).
- [] Первоначальная настройка имени пользователя и email через `git config`.
- [] Создание нового репозитория (`git init`) и проверка его текущего состояния (`git status`).
- [] Перенос изменений в индекс (`git add`) и создание фиксации (`git commit -m`).
- [] Использование файла `.gitignore` для исключения мусорных файлов (кэш, виртуальные окружения, логи).
- [] Просмотр логов разработки (`git log --oneline`).

1.2 Глоссарий

- **Репозиторий (Repository)** – хранилище вашего проекта, содержащее все файлы, каталоги и полную историю изменений с метаданными.
- **Коммит (Commit)** – снимок (snapshot) состояния проекта в определенный момент времени, имеющий уникальный хэш идентификатор.
- **Индекс (Staging Area / Index)** – промежуточная зона, куда файлы добавляются перед коммитом для подготовки к фиксации.

1.3 Полезные ссылки

- Официальная документация: [Git Book на русском](#)
- Интерактивный тренажер: [Learn Git Branching](#)

1.4 Задача на проверку

"Первый слепок проекта"

Инициализируйте пустую папку как Git-репозиторий. Создайте файл `index.html` с произвольным кодом и файл `secret.log`. Сделайте так, чтобы файл `secret.log` игнорировался системой. Добавьте `index.html` в индекс, зафиксируйте изменения с осмысленным комментарием и убедитесь через историю логов, что коммит успешно создан.

Модуль 2: Ветвление и Git Flow

Что нужно уметь: Создавать изолированные ветки для разработки новых фич, безопасно переключаться между ними, выполнять слияние и разрешать возникающие конфликты.

2.1 Чек-лист знаний

- [] Понимание концепции ветвления как механизма создания независимых линий разработки.
- [] Создание новых веток (`git branch`) и переключение на них (`git checkout` или `git switch`).
- [] Быстрое создание с переключением в одну команду (`git checkout -b`).
- [] Слияние веток через `git merge` и понимание разницы между Fast-forward и Three-way merge.
- [] Алгоритм разрешения конфликтов слияния (Conflict Resolution) в коде.
- [] Удаление локальных веток после успешного слияния (`git branch -d`).

2.2 Глоссарий

- **Ветка (Branch)** – указатель на один из коммитов, представляющий собой независимую линию разработки в репозитории.
- **HEAD** – специальный указатель, который ссылается на текущую активную ветку или коммит, с которым вы сейчас работаете.
- **Конфликт слияния (Merge Conflict)** – ситуация, когда в двух сливаемых ветках были изменены одни и те же строки одного файла, и Git не может автоматически определить, какую версию сохранить.

2.3 Полезные ссылки

- Документация: [Ветвление в Git](#)

2.4 Задача на проверку

"Разработчик фичи"

Находясь в ветке `main`, создайте ветку `feature-login` и переключитесь на нее. Создайте файл `auth.py`, напишите в нем

любой код и сделайте коммит. Вернитесь в ветку `main`, создайте файл `auth.py` с другим содержимым в той же строке и сделайте коммит. Попробуйте слить `feature-login` в `main`, искусственно вызовите конфликт слияния, разрешите его вручную и завершите слияние.

Модуль 3: Удаленные репозитории и GitHub

Что нужно уметь: Связывать локальный репозиторий с удаленным сервером GitHub, отправлять свои изменения, забирать обновления коллег и работать по методологии Pull Request.

3.1 Чек-лист знаний

- [] Клонирование существующих удаленных репозиториях с помощью `git clone`.
- [] Добавление ссылки на удаленный репозиторий (`git remote add origin`).
- [] Отправка локальных коммитов на сервер (`git push`) и первичная привязка ветки (`-u origin`).
- [] Получение обновлений с сервера без слияния (`git fetch`) и с автоматическим слиянием (`git pull`).
- [] Механика работы с Pull Request (PR) на GitHub: создание, ревью кода, обсуждение, мёрж.
- [] Понимание концепции форков (Fork) для работы с чужими open-source проектами.

3.2 Справочная таблица основных команд

Команда	Что делает	Пример использования
<code>git clone <url></code>	Скачивает удаленный репозиторий на ваш ПК	<code>git clone https://github.com/user/repo.git</code>
<code>git remote -v</code>	Показывает список привязанных удаленных репозиториях	<code>git remote -v</code>

<code>git push <remote> <branch></code>	Отправляет локальные коммиты на удаленный сервер	<code>git push origin main</code>
<code>git pull <remote> <branch></code>	Скачивает изменения с сервера и сливает их с текущей веткой	<code>git pull origin main</code>
<code>git fetch <remote></code>	Скачивает метаданные и новые коммиты с сервера (без слияния)	<code>git fetch origin</code>

3.3 Полезные ссылки

- Документация: [Работа с удаленными репозиториями](#)
- Руководство: [GitHub Skills интерактивные курсы](#)

3.4 Задача на проверку

"Синхронизация с облаком"

Создайте публичный пустой репозиторий на GitHub. Привяжите ваш локальный репозиторий к удаленному под именем `origin`.

Переименуйте текущую ветку в `main` (если она называется иначе) и отправьте коммиты на GitHub с помощью команды `git push`.

Убедитесь, что ваш код появился в веб-интерфейсе GitHub.

СБОРНИК ПРОВЕРОЧНЫХ ТЕСТОВ

Тест 1: Локальный Git

Часть А: Выберите один правильный ответ

1. Какая команда используется для первой инициализации Git-репозитория в папке проекта?
 - а) `git start`
 - б) `git create`
 - в) `git init`
 - г) `git new`
2. В каком состоянии (зоне) находится файл, если изменения в нем сохранены на диске, но еще не подготовлены для коммита?
 - а) Staged (Индекс)
 - б) Modified / Unstaged (Рабочая директория)
 - в) Committed (Репозиторий)
 - г) Untracked (Неотслеживаемый)
3. Какая команда переносит изменения из рабочей директории в зону индекса (Staging Area)?
 - а) `git commit`
 - б) `git save`
 - в) `git add`
 - г) `git push`
4. Что произойдет, если запустить команду `git commit` без флага `-m`?

- а) Коммит создастся автоматически с пустой строкой описания.
- б) Git откроет текстовый редактор по умолчанию для ввода многострочного сообщения коммита.
- в) Команда упадет с критической ошибкой синтаксиса.
- г) Файлы удалятся из индекса.

5. Как называется специальный файл, в котором прописываются правила исключения файлов и папок из отслеживания Git?

- а) `gitconfig`
- б) `.gitignore`
- в) `exclude.txt`
- г) `.gitkeep`

Часть Б: Задания со свободным ответом (команды)

- 6. Напишите команду глобальной настройки вашего email, которая будет привязываться к коммитам.
- 7. Напишите команду, которая добавляет абсолютно все измененные и новые файлы текущей папки в индекс.
- 8. Исправьте ошибку в синтаксисе коммита: `git commit "Added new layout"`
- 9. Какая команда выводит краткую историю коммитов, где каждый коммит занимает ровно одну строчку?
- 10. Напишите команду для проверки текущего статуса репозитория (показывает, какие файлы изменены, какие в индексе).

Тест 2: Ветвление и конфликты

Часть А: Выберите один правильный ответ

1. Что такое ветка в системе контроля версий Git?
 - а) Полная копия папки проекта на жестком диске.
 - б) Простой перемещаемый указатель на один из коммитов.
 - в) Архивный zip-файл старой версии кода.
 - г) Ссылка на удаленный сервер GitHub.
2. Какая команда позволяет безопасно переключиться на уже существующую ветку с именем `develop`?
 - а) `git switch develop` (или `git checkout develop`)
 - б) `git move develop`
 - в) `git branch develop`
 - г) `git go develop`
3. Что делает команда `git checkout -b feature-api`?
 - а) Удаляет ветку `feature-api`.
 - б) Создает новую ветку `feature-api` и сразу переключает на нее указатель HEAD.
 - в) Показывает лог изменений только для ветки `feature-api`.
 - г) Выполняет слияние ветки с `main`.
4. В какую ветку обольются изменения, если вы находитесь в ветке `main` и вводите команду `git merge feature`?
 - а) Изменения из `main` перейдут в `feature`.
 - б) Изменения из `feature` перейдут в `main`.
 - в) Создастся третья ветка, объединяющая обе.

г) Ничего не произойдет, команда написана неверно.

5. Что означает ситуация Merge Conflict в Git?

а) На жестком диске закончилось свободное место во время слияния.

б) Git обнаружил изменения в одних и тех же строках файла в разных ветках и требует ручного выбора верной версии.

в) У вас нет прав доступа для слияния веток.

г) Код содержит синтаксическую ошибку и не компилируется.

Часть Б: Задания со свободным ответом (команды)

6. Напишите команду, которая выводит список всех существующих локальных веток в репозитории.

7. Напишите команду для удаления ветки `bugfix-tabs`, если она уже была успешно слита с основной веткой.

8. Каким символом (или набором символов) Git размечает границы конфликта в файле, показывая текущие изменения локальной ветки (`<<<<<< HEAD`)? Напишите этот маркер.

9. Вы обнаружили, что находитесь в состоянии «отсоединенного HEAD» (Detached HEAD). Какой командой вернуться назад на ветку `main`?

10. Какую команду нужно выполнить после того, как вы вручную разрешили конфликт слияния в файлах и сохранили их, чтобы продолжить/завершить процесс мёржа?

Тест 3: Работа с GitHub и совместная разработка

Часть А: Выберите один правильный ответ

1. Чем команда `git clone` отличается от `git init`?
 - а) `git clone` работает только без интернета.
 - б) `git init` создает новый пустой репозиторий, а `git clone` копирует уже существующий удаленный репозиторий со всей историей.
 - в) `git clone` скачивает только один файл.
 - г) Это абсолютно одинаковые команды.
2. Какая команда используется для отправки ваших локально зафиксированных коммитов на удаленный репозиторий GitHub?
 - а) `git pull`
 - б) `git upload`
 - в) `git push`
 - г) `git fetch`
3. В чем главное отличие команды `git pull` от команды `git fetch`?
 - а) `git pull` работает быстрее.
 - б) `git fetch` только скачивает новые данные с сервера, но не сливает их с локальным кодом; `git pull` скачивает и сразу делает merge.
 - в) `git pull` удаляет файлы с сервера, а `git fetch` закачивает.
 - г) `git fetch` используется только на Windows.
4. Что такое Pull Request (PR) на GitHub?

- а) Команда в терминале для скачивания кода.
- б) Предложение разработчика влить его изменения из одной ветки в другую ветку основного репозитория с возможностью ревью кода.
- в) Сообщение об ошибке в системе.
- г) Специальный вид коммита.

5. Как по умолчанию называется главный удаленный репозиторий-источник, который автоматически создается при клонировании?

- а) `server`
- б) `main`
- в) `origin`
- г) `remote-hub`

Часть Б: Задания со свободным ответом (команды)

- 6. Напишите команду, которая связывает ваш локальный репозиторий с удаленным репозиторием на GitHub по URL: `[https://github.com/user/demo.git](https://github.com/user/demo.git)` под именем `origin`.
- 7. Напишите команду отправки локальной ветки `develop` на удаленный сервер `origin`.
- 8. Как проверить список всех привязанных к проекту адресов удаленных репозитория (вместе с их URL)? Напишите команду с флагом.
- 9. Вы хотите внести вклад в чужой проект, к которому у вас нет прямого доступа на запись. Какую кнопку в веб-интерфейсе GitHub нужно нажать, чтобы скопировать репозиторий к себе в аккаунт?
- 10. Напишите команду, которая скачивает обновления из удаленной ветки `main` сервера `origin` и сразу пытается применить их к вашей текущей ветке.

ОТВЕТЫ И РАЗБОР ЗАДАНИЙ

Тест 1: Локальный Git

- 1: в) `git init` инициализирует новый пустой Git-репозиторий в текущем каталоге.
- 2: б) Файл находится в рабочей директории в состоянии Modified, он еще не добавлен в индекс.
- 3: в) `git add` добавляет файлы в индекс для подготовки к следующему коммиту.
- 4: б) Без флага сообщения Git принудительно откроет текстовый редактор (например, Vim или Nano), чтобы вы написали комментарий.
- 5: б) `.gitignore` управляет исключениями трекинга файлов.
- 6: `git config --global user.email "your_email@example.com"`
- 7: `git add .` (или `git add -A`)
- 8: Пропущен флаг `-m` перед сообщением. Правильно: `git commit -m "Added new layout"`
- 9: `git log --oneline`
- 10: `git status`

Тест 2: Ветвление и конфликты

1: 6) Ветка в Git – это всего лишь легковесный указатель на конкретный коммит, а не физическая копия файлов проекта.

2: а) `git switch develop` или `git checkout develop` меняет активную ветку.

3: 6) Флаг `-b` указывает Git создать ветку перед тем, как переключить на нее HEAD.

4: 6) Изменения вливаются в *текущую* ветку (в которой вы стоите) из *указанной* в аргументе.

5: 6) Конфликт возникает, когда автоматический мёрж невозможен из-за пересекающихся правок в кодовой базе.

6: `git branch`

7: `git branch -d bugfix-tabs`

8: `<<<<<<<` (7 знаков "меньше").

9: `git switch main` или `git checkout main`

10: Нужно добавить исправленные файлы в индекс с помощью `git add <file>`, а затем выполнить `git commit`.

Тест 3: Работа с GitHub и совместная разработка

1: 6) `git clone` скачивает готовый проект из удаленного облака, разворачивая локальную копию.

2: в) `git push` "выталкивает" локальную историю коммитов на сервер.

3: 6) `git fetch` – это безопасный режим просмотра обновлений. `git pull` объединяет `fetch` и `merge`.

4: 6) Pull Request – это основной инструмент код-ревью и коллаборации на платформах вроде GitHub.

5: в) `origin` – стандартное имя по умолчанию для основного удаленного сервера.

6: `git remote add origin`
`[https://github.com/user/demo.git](https://github.com/user/demo.git)`

7: `git push origin develop`

8: `git remote -v`

9: Кнопку **Fork**.

10: `git pull origin main`